Data Cleaning

```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
──
✔ dplyr     1.1.4     ✔ readr     2.1.5
✔ forcats   1.0.1     ✔ stringr   1.5.2
✔ ggplot2   4.0.0     ✔ tibble    3.3.0
✔ lubridate 1.9.4     ✔ tidyr     1.3.1
✔ purrr     1.1.0
── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
g24_sov_precinct <- read_csv("~/stat133/gerrymandering-leyna-liu/data/
g24_sov_by_g24_svprec.csv")
```

```
Rows: 51123 Columns: 76
── Column specification
────────────────────────────────────────────────────────
Delimiter: ","
chr (49): FIPS, SVPREC, SVPREC_KEY, ELECTION, GEO_TYPE, ASSAIP01,
ASSDEM01, ...
dbl (27): COUNTY, ADDIST, CDDIST, SDDIST, BEDIST, TOTREG, DEMREG, REPREG,
AI...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
# Using glimpse gives us an idea of each column and the elements within each
column.

# As background, each row represents one precinct, and each column represents
something new. So, COUNTY -> for example is county ID number; FIPS -> is a 5
digit county code; SVPREC -> the statewide precinct code or basically a
precinct ID inside a single county (multiple precincts in a county THAT REPEAT
```

```
IN OTHER COUNTIES TOO, so think of it as a house address that is not
particular to one "city"); ADDIST -> assembly district number; SVPREC_KEY ->
statewide key that is unique, unlike SVPREC it is a unique code combining
county FIPS code, SVPREC, and election code, so each SVPREC_KEY should be
unique per row; DEMREG -> registration count for democrats; REPREG ->
registration counts for Republicans ; AIPREG -> registration count for
American Independent; GRNREG -> registration count for green party.

glimpse(g24_sov_precinct)
```

```
Rows: 51,123
Columns: 76
$ COUNTY     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,…
$ FIPS       <chr> "06001", "06001", "06001", "06001", "06001", "06001",
"0600…
$ SVPREC     <chr> "200100", "200100A", "200200", "200200A", "201400",
"201400…
$ ADDIST     <dbl> 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 14,
14,…
$ SVPREC_KEY <chr> "06001200100", "06001200100A", "06001200200",
"06001200200A…
$ ELECTION   <chr> "g24", "g24", "g24", "g24", "g24", "g24", "g24", "g24",
"g2…
$ GEO_TYPE   <chr> "svprec", "svprec", "svprec", "svprec", "svprec",
"svprec",…
$ CDDIST     <dbl> 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12,
12,…
$ SDDIST     <dbl> 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7,
7,…
$ BEDIST     <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,…
$ TOTREG     <dbl> 3535, 0, 2442, 0, 3773, 0, 541, 0, 1105, 0, 948, 0, 2721,
0…
$ DEMREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ REPREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ AIPREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ GRNREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ LIBREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ NLPREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ REFREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```
0,…
$ DCLREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ MSCREG     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ TOTVOTE    <dbl> 256, 2804, 262, 1816, 283, 2782, 89, 343, 394, 297, 837,
29…
$ DEMVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ REPVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ AIPVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ GRNVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ LIBVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ NLPVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ REFVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ DCLVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ MSCVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ PRCVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ ABSVOTE    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,…
$ ASSAIP01   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ ASSDEM01   <chr> "94", "444", "117", "348", "107", "588", "45", "105",
"181"…
$ ASSDEM02   <chr> "110", "2023", "91", "1243", "128", "1841", "24", "172",
"1…
$ ASSREP01   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ ASSREP02   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ CNGDEM01   <chr> "102", "1668", "108", "1063", "139", "1688", "35", "192",
"…
$ CNGDEM02   <chr> "102", "771", "99", "513", "98", "739", "38", "93", "143",
…
$ CNGIND01   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ CNGREP01   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ CNGREP02   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
```

```
"0",…
$ PRSAIP01   <chr> "3", "10", "1", "6", "3", "13", "2", "2", "3", "4", "5",
"2…
$ PRSDEM01   <chr> "181", "2562", "207", "1647", "231", "2522", "73", "297",
"…
$ PRSGRN01   <chr> "9", "48", "13", "41", "9", "52", "0", "13", "6", "13",
"23…
$ PRSLIB01   <chr> "1", "10", "2", "3", "4", "13", "0", "0", "3", "2", "3",
"0…
$ PRSPAF01   <chr> "5", "17", "7", "12", "8", "32", "2", "4", "3", "7", "16",
…
$ PRSREP01   <chr> "51", "108", "26", "83", "17", "111", "11", "23", "55",
"24…
$ PR_2_N     <chr> "58", "493", "45", "342", "39", "399", "17", "45", "52",
"3…
$ PR_2_Y     <chr> "169", "2156", "196", "1385", "226", "2231", "66", "278",
"…
$ PR_32_N    <chr> "78", "636", "55", "439", "74", "536", "14", "60", "81",
"6…
$ PR_32_Y    <chr> "148", "1966", "187", "1261", "190", "2070", "68", "255",
"…
$ PR_33_N    <chr> "136", "1774", "105", "1092", "124", "1509", "30", "127",
"…
$ PR_33_Y    <chr> "86", "784", "126", "584", "133", "1053", "49", "177",
"231…
$ PR_34_N    <chr> "123", "1485", "121", "1027", "144", "1515", "46", "186",
"…
$ PR_34_Y    <chr> "98", "980", "105", "601", "96", "941", "33", "107",
"174",…
$ PR_35_N    <chr> "54", "581", "45", "419", "58", "563", "20", "57", "61",
"5…
$ PR_35_Y    <chr> "171", "2003", "188", "1261", "196", "1988", "58", "248",
"…
$ PR_36_N    <chr> "106", "1356", "142", "888", "146", "1487", "49", "197",
"2…
$ PR_36_Y    <chr> "118", "1223", "99", "786", "119", "1084", "31", "112",
"14…
$ PR_3_N     <chr> "51", "133", "25", "116", "33", "152", "10", "26", "38",
"2…
$ PR_3_Y     <chr> "183", "2553", "220", "1646", "240", "2508", "74", "295",
"…
$ PR_4_N     <chr> "52", "381", "37", "271", "37", "330", "14", "41", "40",
"2…
$ PR_4_Y     <chr> "181", "2294", "209", "1472", "231", "2316", "68", "279",
"…
$ PR_5_N     <chr> "94", "961", "66", "605", "63", "742", "19", "76", "72",
"6…
$ PR_5_Y     <chr> "132", "1660", "168", "1096", "197", "1862", "61", "240",
```

```
"…
$ PR_6_N     <chr> "75", "607", "59", "407", "57", "532", "17", "53", "85",
"5…
$ PR_6_Y     <chr> "143", "1958", "180", "1274", "196", "2029", "62", "257",
"…
$ SENDEM01   <chr> "107", "1719", "101", "1102", "136", "1578", "37", "153",
"…
$ SENDEM02   <chr> "103", "809", "114", "516", "105", "908", "34", "133",
"174…
$ SENREP01   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ SENREP02   <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
"0",…
$ USPDEM01   <chr> "172", "2461", "199", "1572", "217", "2444", "67", "285",
"…
$ USPREP01   <chr> "53", "155", "34", "111", "32", "153", "11", "23", "53",
"3…
$ USSDEM01   <chr> "173", "2487", "207", "1593", "222", "2478", "67", "288",
"…
$ USSREP01   <chr> "55", "155", "29", "109", "33", "151", "12", "23", "51",
"2…
```

```
# Now, check for repeated precincts, using the column SVPREC_KEY, as each
SVREC_KEY should be unique to each row (which each row represents a precinct).

# duplicated() checks row by row to see if a element has repeated in
SVPREC_KEY column, and marks TRUE if it has and FALSE if it hasn't repeated.

# By taking the sum of duplicated, it checks how many TRUEs and FALSEs (0s)
there are, so output of 0 means no repeats, output of >0 means there're TRUEs
(1s) or repeats.

sum(duplicated(g24_sov_precinct$SVPREC_KEY))
```

```
[1] 0
```

```
# Just to make sure I am making certain columns numeric variables of it isn't
already.

g24_sov_precinct <- g24_sov_precinct |>
    mutate(
        COUNTY = as.numeric(COUNTY),
        FIPS = as.numeric(FIPS),
        ADDIST = as.numeric(ADDIST),
        CDDIST = as.numeric(CDDIST),
```

```
        SDDIST = as.numeric(SDDIST),
        BEDIST = as.numeric(BEDIST),
        TOTREG = as.numeric(TOTREG),
        TOTREG = as.numeric(TOTREG),
    )

# In fact, since there are way too many columns to type them each one by one
as.numeric(), I will use across() function.

# across() tells R to apply a command to more than 1 row at once

g24_sov_precinct <- g24_sov_precinct |>
    mutate(
        across(32:76, as.numeric)
    )
```

```
Warning: There were 44 warnings in `mutate()`.
The first warning was:
ℹ In argument: `across(32:76, as.numeric)`.
Caused by warning:
! NAs introduced by coercion
ℹ Run `dplyr::last_dplyr_warnings()` to see the 43 remaining warnings.
```

```
view(g24_sov_precinct)
```

```
# Make sure there are no missing congressional district numbers (for each
precinct):
sum(is.na(g24_sov_precinct$CDDIST))
```

```
[1] 0
```

```
# is.na() gives the TRUEs and FALSEs (with FALSE being no NA), and the sum
should be 0.
```

```
# we are going to save the new cleaned dataset into the data section of my
stat133 folder on my computer
write_csv(g24_sov_precinct, "~/stat133/gerrymandering-leyna-liu/data/
g24_sov_precinct.csv")

# write_csv() allows you to add data in csv format to your folder
# general formula: write_csv(x, "y")
# where x = the data frame you want to save into your folder
```

```
# "y" = path to get to file/folder you want to save it at, and at the end add
the name of the new file: g24_sov_precinct.csv
```

Part 5: Re-running the 2024 Election

```
library(sf)
```

```
Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(tidyverse)
```

```
g24_sr_precinct <- st_read("~/stat133/gerrymandering-leyna-liu/data/
shapefiles/srprec_state_g24_v01_shp.shp")
```

```
Reading layer `srprec_state_g24_v01_shp' from data source
  `/Users/leynaliu/stat133/gerrymandering-leyna-liu/data/shapefiles/
srprec_state_g24_v01_shp.shp'
  using driver `ESRI Shapefile'
```

```
Warning in CPL_read_ogr(dsn, layer, query, as.character(options), quiet, :
GDAL
Message 1:
/Users/leynaliu/stat133/gerrymandering-leyna-liu/data/shapefiles/
srprec_state_g24_v01_shp.shp
contains polygon(s) with rings with invalid winding order. Autocorrecting
them,
but that shapefile should be corrected using ogr2ogr for example.
```

```
Simple feature collection with 24224 features and 6 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -124.482 ymin: 32.52883 xmax: -114.1312 ymax: 42.0095
Geodetic CRS:  NAD83
```

```
g24_proposed_congressional_map <- st_read("~/stat133/gerrymandering-leyna-liu/
data/shapefiles/AB604.shp")
```

```
Reading layer `AB604' from data source
  `/Users/leynaliu/stat133/gerrymandering-leyna-liu/data/shapefiles/
AB604.shp'
```

```
  using driver `ESRI Shapefile'
Simple feature collection with 52 features and 15 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -13857270 ymin: 3832931 xmax: -12705030 ymax: 5162404
Projected CRS: WGS 84 / Pseudo-Mercator
```

```
g24_sov_precinct <- read_csv("~/stat133/gerrymandering-leyna-liu/data/
g24_sov_precinct.csv")
```

```
Rows: 51123 Columns: 76
── Column specification
────────────────────────────────────────────────────────
Delimiter: ","
chr  (4): SVPREC, SVPREC_KEY, ELECTION, GEO_TYPE
dbl (72): COUNTY, FIPS, ADDIST, CDDIST, SDDIST, BEDIST, TOTREG, DEMREG,
REPR...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
# First we have to clean the data (SR precincts)

sr_shp <- g24_sr_precinct |>
    st_transform(3310) |>
    st_set_precision(1) |>
    st_make_valid() |>
    st_collection_extract("POLYGON")

# st_transform(3310) -> changes the CRS (coordinate reference system) of the
different shapes to EPSG 3310 (which is a equal area projection for
California) -> so essentially coordinates are adjusted to the 3310 system for
the shapes

# st_set_precision(1) -> sets precision of coordinates to 1 -> helps mitigate
numerical errors when overlapping shapes, cleans up the coordinates

# st_make_valid() -> adjusts invalid shapes

# st_collection_extract("POLYGON") -> keeps only polygon geometries -> for ex.
if there were mixed geometry types like multipolygons or multilines, only
polygon would be extracted
```

```
# Now, I will repeat the exact same process for the proposed congressional map
data

proposed_map <- g24_proposed_congressional_map |>
    st_transform(3310) |>
    st_set_precision(1) |>
    st_make_valid() |>
    st_collection_extract("POLYGON")
```

```
# Now we want to find where our precincts overlap with the districts. For
example, Precinct A could lie partially in District 1, but also partially in
District 2.

# st_intersection() is able to create a table of those different overlaps like
how one chunk of precinct A is in district 1, and another chunk of precinct A
is in district 2 (which in the table will translate to 2 seperate rows for
each chunk of precinct A)

overlapping <- st_intersection(
    sr_shp |>
        select(SRPREC),
    proposed_map |>
        select(new_dist = DISTRICT)
) |>
    mutate(overlap_area = st_area(geometry)) |>
    group_by(SRPREC) |>
    mutate(prop_of_precinct = as.numeric(overlap_area / sum(overlap_area)))
```

```
Warning: attribute variables are assumed to be spatially constant throughout
all geometries
```

```
# st_intersection() takes two spatial objects (like our two polygons here),
and find where they overlap. the new output will also have new geometries for
every overlapping piece

# Next, from our sr_shp polygon we want to select only the SRPREC column,
which is the precinct IDs.

# Then, from our proposed_map polygon we select only the DISTRICT column,
which is the district IDs, and simultaneously also renaming the column to
new_dist

# Next, we create a new column called overlap_area, by using st_area()
function which calculates the area of each geometry/polygon. geometry is the
column that stores the actual shape/polygon (remember from lecture it stores
```

```
the points that make up the polygon) of each overlapping piece.

# Now, we group by SRPREC (precinct IDs) -> telling R to treat rows with the
same precinct IDs as 1 group

# the previous grouping prepares us to make new column called pct_precinct,
where we take area of overlapping piece in precinct divided by total precinct
area -> essentially asking what proportion of precinct in this district
# as.numeric() is important since the variables/columns have not been
converted to numeric variable yet

# For ex. if precinct A has 2 polygons/pieces:
# 1 piece in District 1: area = 30
# 1 piece in District 2: area = 70
# sum of overlap areas = 100
# -> piece 1 = 30/100 = 0.3
# -> piece 2 = 70/100 = 0.7
```

```
# First, dataset g24_sov_precinct does NOT have column SRPREC, instead it only
has column SVPREC. So we need to rename it to SRPREC, so that it can be joined
by overlapping dataset that has SRPREC column.

# So I select SVPREC, CNGDEM01, CNGREP01 from g24_sov_precinct and rename them

# Then, I can finally join the dataframes overlapping and votes_precinct by
common key SRPREC.

# So, after joining by key SRPREC, we create new columns d, taking d_votes,
the total amount of democratic votes in the whole precinct (from
votes_precinct dataset), and multiplying it by the proportion of area of
precinct in a district (our column from overlapping dataset) -> this tells us
how many democratic votes should we assign to this precinct based on how much
of the area is in a certain district

# for ex. if precinct 10 has 200 democratic votes, and 70% of this precinct is
in district 7 (with the other 30% of this precinct in district 2), then
200*0.70 = 140 votes should be assigned to district 7 from this specific
presinct (precinct 10).

votes_precinct <- g24_sov_precinct |>
    select(SVPREC, CNGDEM01, CNGREP01) |>
    rename(
        SRPREC = SVPREC,
        d_votes = CNGDEM01,
        r_votes = CNGREP01
    )
```

```
joined_precinct <- overlapping |>
    left_join(votes_precinct, by = "SRPREC") |>
    mutate(
        d_votes_based_district = d_votes * prop_of_precinct,
        r_votes_based_district = r_votes * prop_of_precinct
    )
```

```
Warning in sf_column %in% names(g): Detected an unexpected many-to-many
relationship between `x` and `y`.
ℹ Row 94 of `x` matches multiple rows in `y`.
ℹ Row 20750 of `y` matches multiple rows in `x`.
ℹ If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.
```

```
# So first you group_by(new_dist) column from joined_precinct dataset, which
tells R to treat rows with the same district number as one group. We want to
later sum the votes in each district.

# Now, we calculate the summary statistics, so we sum all the democratic votes
assigned to each area based on the district. For ex. if District 7 is made up
of 17 different areas/pieces, then, this sums all the votes for each piece,
resulting in total votes in that district for democrats. Then, I just repeat
the process for Republicans. So for each district we sum up all the areas that
make up each district, resulting in total republican votes cast in that
district.

# finally, we create a winner column, so if d_votes_total_dist >
r_votes_total_dist, then print "D", otherwise print "R". Then, we create
column total which adds d_votes_total_dist and r_votes_total_dist. Then,
d_prop is proportion of d votes per district, and r_prop is proporiton of r
votes per district.

final <- joined_precinct |>
    group_by(new_dist) |>
    summarize(
        d_votes_total_dist = sum(d_votes_based_district, na.rm = TRUE),
        r_votes_total_dist = sum(r_votes_based_district, na.rm = TRUE)
    ) |>
        mutate(
            winner = if_else(d_votes_total_dist > r_votes_total_dist, "D",
"R"),
            total = d_votes_total_dist + r_votes_total_dist,
            d_prop = d_votes_total_dist / total,
            r_prop = r_votes_total_dist / total
        )
```

```r
# I am saving the final dataset in my data folder under gerrymandering-leyna-liu so I can use it in PART 6 of project

write_csv(final, "data/final.csv")
```